

Politecnico di Torino

Master's Degree in Computer Engineering

Migrating Legacy Services to a Cloud-Native Architecture: A Case Study on Outsight Cloud



**Politecnico
di Torino**

Supervisor: Prof. Fulvio Giovanni Risso

Candidate: Giovanni Russo

Matricola: s343424

Academic Year 2025–2026

Abstract

This thesis explores the migration of a legacy cloud platform toward a cloud-native architecture, using the Outsight Cloud platform as a case study. The existing system relied heavily on Airtable-based data management, manual operational processes, and infrastructure decisions that increasingly limited scalability, maintainability, and long-term sustainability.

The work focuses on how to modernize the platform while maintaining service continuity and reducing both technical and operational risks during the transition. Different solutions for identity management, hosting, storage, and data persistence are analyzed and compared. The proposed architecture is based on managed AWS services, Infrastructure as Code with Terraform, and a gradual migration from Airtable to PostgreSQL, supported by coexistence mechanisms designed to minimize disruption.

A key contribution of the thesis is a practical migration methodology that connects architectural decisions with measurable technical and economic criteria. The analysis considers aspects such as security, backward compatibility, observability, testing, deployment reliability, infrastructure costs, and operational overhead.

The thesis also presents the migration foundations already implemented within the project, including CI pipelines, smoke testing for dual backends, data replication and validation scripts, and separated infrastructure environments. The evaluation therefore focuses on the feasibility of the migration strategy, the quality of the target architecture, and the extent to which the proposed approach reduces migration risk.

The final result is a practical framework for supporting legacy-to-cloud-native migrations in real production environments. The work is intended for engineers and organizations looking to modernize existing systems through structured architectural evolution rather than simple technology replacement.

Acknowledgements

I would like to express my sincere gratitude to Oversight for providing me with the opportunity to complete this internship and work on challenging and meaningful projects. I am particularly thankful to my supervisor and the entire development team for their guidance, support, and patience throughout this journey.

I would also like to thank the Politecnico di Torino for the academic foundation that made this work possible, and my thesis advisor for valuable feedback and encouragement.

Finally, I am deeply grateful to my family and friends for their unwavering support during my studies.

Contents

Abstract	I
Acknowledgements	II
List of Figures	VII
List of Tables	VIII
1 Introduction	1
1.1 Industrial Context and Legacy Constraints	1
1.2 Problem Statement	2
1.3 Thesis Scope	2
1.4 Thesis Structure	3
2 Background and Related Work	5
2.1 Cloud-Native Principles for Legacy Modernization	5
2.1.1 Incremental Delivery and Operability	5
2.1.2 Infrastructure as Code	6
2.2 Migration Strategies and Patterns	6
2.2.1 Cloud Migration Fundamentals	6
2.2.2 Coexistence as a First-Class Requirement	8
2.2.3 Decision-Driven Migration	8
2.3 IAM Modernization in Legacy Systems	8
2.3.1 From Application-Coupled Identity to Standard Protocols	8
2.3.2 Open-Source and Managed IAM Trade-offs	9
2.4 Limitations in Existing Approaches	9

2.4.1	Fragmented Treatment of Technical and Economic Criteria . . .	9
2.4.2	Limited Evidence on Incremental Validation	9
2.5	Positioning of This Thesis	9
3	Case Study Baseline (As-Is System)	11
3.1	Current Architecture and Data Flow	11
3.2	Airtable as Primary Data Storage	12
3.3	Limitations of Airtable	12
3.3.1	Data Modeling and Consistency	13
3.3.2	Limited Relational Capabilities	13
3.3.3	Operational Inefficiency	13
3.3.4	Scalability, Coupling, and Cost	13
3.4	Motivation for Migration to PostgreSQL	13
4	Migration Methodology and Decision Framework	15
4.1	Migration Goals and Success Criteria	15
4.2	Candidate Solutions Considered	16
4.2.1	Infrastructure and Runtime Options	16
4.2.2	Alternative Cloud Platforms Considered	16
4.2.3	Data-Layer Migration Options	18
4.2.4	Identity Evolution Options	18
4.3	Technical and Economic Evaluation Criteria	18
4.3.1	Technical Criteria	18
4.4	Selected Strategy and Rationale	19
4.5	Risk Management and Coexistence Plan	20
5	Target Architecture and Solution Design (To-Be)	21
5.1	Initial Architecture of Outsight Cloud	21
5.1.1	Client Layer	22
5.1.2	Reverse Proxy Layer	23
5.1.3	Application Layer	23
5.1.4	External Services Layer	23
5.1.5	Architectural Limitations	24
5.2	Architecture Overview	24

5.3	Client and Delivery Layer	26
5.3.1	Browser	26
5.3.2	Amazon CloudFront	26
5.4	Load Balancing Layer	26
5.4.1	Application Load Balancer	26
5.5	Application Layer	26
5.5.1	ECS Fargate Runtime	26
5.5.2	Frontend Service (Vue)	27
5.5.3	Backend Service (Node.js API)	27
5.6	Managed Services Layer	27
5.6.1	PostgreSQL	27
5.6.2	Redis	27
5.6.3	Amazon S3	27
5.6.4	Amazon Cognito	28
5.6.5	Amazon SES	28
5.6.6	Amazon SNS	28
5.7	Architectural Benefits	28
6	Migration Implementation	29
6.1	Incremental Execution Plan	29
6.2	Infrastructure as Code and Environment Separation	30
6.2.1	Infrastructure as Code (IaC)	30
6.2.2	Terraform-Driven Provisioning	30
6.2.3	Terraform Workflow: Write, Plan, Apply	31
6.2.4	State Management and Team Collaboration	31
6.2.5	Why Terraform in the Outsight Cloud Migration	32
6.2.6	Environment Governance	32
6.3	Data Migration Tooling and Validation	32
6.3.1	Relational Foundation	32
6.3.2	Replication and Validation Scripts	33
6.4	CI/Test Strategy for Dual-Backend Consistency	33
6.4.1	Dual-Backend Test Design	33
6.4.2	Containerized Validation Environment	33
6.4.3	Pipeline Integration	33

6.5	Implementation Scope and Current Status	34
7	Evaluation and Discussion	35
7.1	Validation Methodology	35
7.2	Technical Results	35
7.2.1	Infrastructure and Deployment Readiness	35
7.2.2	Data Migration Readiness	36
7.2.3	Testing and Verification Readiness	36
7.3	Migration KPIs and Economic Impact	36
7.3.1	Technical Migration KPIs	36
7.3.2	Economic Migration KPIs	36
7.4	Limitations and Lessons Learned	37
7.4.1	Limitations	37
7.4.2	Lessons Learned	37
8	Conclusions and Future Work	39
8.1	Key Outcomes	39
8.2	Future Improvements	39

List of Figures

3.1	Outsight Cloud baseline architecture before the migration, as documented in the pre-migration technical material.	12
5.1	Initial architecture of Outsight Cloud before AWS migration.	22
5.2	Final AWS architecture adopted for Outsight Cloud.	25

List of Tables

4.1 Comparative assessment of cloud provider alternatives for Oversight
Cloud migration 16

Chapter 1

Introduction

1.1 Industrial Context and Legacy Constraints

In recent years, cloud computing has progressively transformed how software systems are designed, deployed, and maintained. Organizations increasingly migrate applications from traditional infrastructures to cloud-native environments to improve scalability, maintainability, automation capabilities, and operational efficiency.

However, migrating an existing platform to the cloud is not a simple technology replacement activity. Legacy systems typically evolve over multiple years through incremental additions of services, integrations, and operational procedures. As a consequence, they embed architectural dependencies, technology constraints, and operational practices that make migration non-trivial.

This thesis analyzes Outsight Cloud (OC), a platform used to manage LiDAR-related workflows and assets, including organizations, sites, simulations, documentation, and associated files. Before migration, the platform relied on a Docker Compose-based containerized architecture integrated with several external services. Core application data was managed through Airtable, while additional capabilities depended on external storage systems, documentation engines, and notification services.

1.2 Problem Statement

The engineering problem addressed in this work is the migration of Outsight Cloud from a legacy architecture to a cloud-native environment under real technical and operational constraints.

Although the initial architecture supported rapid development in earlier product stages, several limitations progressively emerged:

- strong dependency on external services;
- lack of robust relational data modeling capabilities;
- operational inefficiencies caused by manual workflows;
- limited scalability as platform usage increased;
- increasing maintenance complexity;
- recurring infrastructure and platform costs;
- absence of cloud-native orchestration mechanisms.

These limitations made the architecture progressively harder to evolve while the platform continued to grow. For this reason, the company initiated a cloudification process aimed at redesigning infrastructure and migrating services to AWS.

The migration objective was not limited to infrastructure relocation. It required redefining the architectural model, selecting appropriate cloud technologies, introducing Infrastructure as Code practices, replacing legacy components, and designing a maintainable target architecture.

1.3 Thesis Scope

The thesis focuses on designing and justifying a migration strategy for a legacy platform under real technical and economic constraints. The scope includes:

- defining the problem context and migration drivers;
- comparing candidate cloud migration options;

- selecting and motivating a target architecture;
- presenting implementation and validation foundations already supported by project evidence.

1.4 Thesis Structure

This thesis is organized as follows:

Chapter 2 introduces the conceptual and related-work foundations on cloud-native modernization, migration patterns, and IAM evolution in legacy systems.

Chapter 3 describes the case-study baseline (as-is architecture), including technical debt and cost-related migration drivers.

Chapter 4 presents the migration methodology and decision framework, including candidate options and selection rationale.

Chapter 5 defines the target architecture (to-be), its main components, and design trade-offs.

Chapter 6 details the incremental migration implementation foundations, with focus on IaC, data migration tooling, and validation pipeline.

Chapter 7 discusses the evaluation approach and current evidence regarding migration feasibility, quality, and impact.

Chapter 8 concludes the thesis by summarizing contributions and outlining future work.

Chapter 2

Background and Related Work

This chapter provides the conceptual background required to frame the migration problem and positions the thesis with respect to related work.

2.1 Cloud-Native Principles for Legacy Modernization

Cloud-native modernization is not limited to container adoption. In a migration context, it also requires reproducible infrastructure, explicit service boundaries, automated validation, and operational observability. These principles are especially relevant when modern and legacy components must coexist during transition.

2.1.1 Incremental Delivery and Operability

A recurring principle in modernization literature is incremental delivery over big-bang replacement. Incremental delivery reduces migration risk, enables early feedback, and supports rollback strategies. To be effective, it requires infrastructure automation, environment consistency, and testable interfaces between old and new components.

2.1.2 Infrastructure as Code

Infrastructure as Code (IaC) is a key enabler for modernization because it turns deployment topology, security controls, and environment configuration into versioned artifacts. This supports traceability, reviewability, and reproducibility across development and production environments.

2.2 Migration Strategies and Patterns

Common migration strategies include rehosting, replatforming, refactoring, and selective rebuilding. In practice, many enterprise projects combine these strategies across subsystems. Decision criteria generally include coupling level, criticality, migration effort, and risk exposure.

2.2.1 Cloud Migration Fundamentals

Cloud migration is the process of moving applications, data, and infrastructure resources from traditional environments to cloud-based infrastructures [1]. In practice, this transition is executed in a controlled and systematic way, with prioritized workloads, staged rollout, and validation activities designed to minimize disruption.

Historically, many organizations operated software systems on self-managed infrastructures hosted in local data centers or virtualized environments. While these architectures provided direct control, they also introduced significant operational complexity, including infrastructure maintenance, capacity planning, hardware life-cycle management, backup strategies, and service availability governance.

Cloud migration therefore should not be interpreted as pure workload relocation. In many cases it also requires architectural redesign decisions to improve scalability, maintainability, automation, and long-term operational efficiency.

Main Drivers and Benefits

Reference guidance highlights recurring migration drivers and benefits [1]:

- **Cost efficiency:** reduced on-premises operational burden and better resource-cost alignment;

- **Scalability and flexibility:** elastic allocation under variable workloads;
- **Reliability and managed operations:** broader use of managed services for availability, monitoring, backup, and recovery;
- **Performance:** access to modern infrastructure capabilities and distributed delivery models;
- **Sustainability:** improved energy efficiency through hyperscale infrastructure optimization.

Migration Strategies

Cloud migration can be approached through multiple strategies, selected according to modernization goals and technical constraints [1]:

- **Rehosting:** move workloads with minimal changes (lift-and-shift);
- **Replatforming:** adapt selected components to better exploit cloud capabilities;
- **Refactoring:** redesign parts of the system to align with cloud-native principles.

Execution Phases and Challenges

Migration programs are often organized in three macro-phases [1]: assess, mobilize, and migrate/modernize. These phases help structure technical discovery, platform preparation, pilot execution, and scaled rollout.

Typical challenges include legacy technical complexity, execution constraints during large-scale migration waves, and cloud-skills gaps in delivery teams [1]. For this reason, successful migration programs combine technical planning, organizational preparation, and incremental verification.

Implication for Oversight Cloud

In the Oversight Cloud case, migration was not limited to infrastructure relocation. It required service redesign, replacement of legacy dependencies, and adoption of

Infrastructure as Code practices. The objective was to evolve from an externally coupled legacy setup toward a more maintainable and scalable cloud-native architecture on AWS.

2.2.2 Coexistence as a First-Class Requirement

For systems that cannot tolerate interruption, coexistence between legacy and target components becomes a design requirement. This introduces additional complexity: contract compatibility, dual-path validation, and progressive switchover mechanisms.

2.2.3 Decision-Driven Migration

Related work often describes migration patterns conceptually, but less frequently provides explicit decision frameworks that combine technical and economic rationale. This gap is relevant for industrial contexts where architecture choices must be justified beyond purely technical preference.

2.3 IAM Modernization in Legacy Systems

Identity and Access Management (IAM) modernization is a recurrent challenge in legacy platforms. Token-based approaches and standards such as OAuth 2.0 and OpenID Connect (OIDC) provide stronger interoperability and security controls than ad-hoc authentication schemes.

2.3.1 From Application-Coupled Identity to Standard Protocols

Legacy systems frequently embed identity assumptions in application code and persistence layers. Migrating to standards-based IAM requires decoupling these assumptions, redefining authorization boundaries, and validating compatibility with existing clients and workflows.

2.3.2 Open-Source and Managed IAM Trade-offs

The choice between open-source IAM solutions and managed identity platforms affects control, integration flexibility, and operating cost. For migration projects, this decision also impacts how quickly existing services can adopt new identity flows.

2.4 Limitations in Existing Approaches

The literature provides strong foundations on migration patterns and IAM technologies, but industrial guidance is often fragmented across separate dimensions.

2.4.1 Fragmented Treatment of Technical and Economic Criteria

Technical architecture decisions are frequently discussed without detailed economic framing. However, recurring infrastructure costs, operational burden, and dependency reduction are often decisive in enterprise migration roadmaps.

2.4.2 Limited Evidence on Incremental Validation

Many case studies report final architectures but provide limited detail on intermediate validation mechanisms, such as dual-backend tests, migration data checks, and environment-separated verification workflows.

2.5 Positioning of This Thesis

This thesis addresses the above limitations through a decision-oriented migration case study. The contribution is not a generic technology survey, but an end-to-end reasoning process from baseline constraints to architecture selection, incremental implementation, and migration-focused validation.

Chapter 3

Case Study Baseline (As-Is System)

This chapter describes the initial state of the Oversight Cloud platform before the migration design was formalized. The goal is to establish the baseline against which migration decisions are evaluated.

3.1 Current Architecture and Data Flow

The as-is platform handled LiDAR-related assets and simulations through a web application and backend API stack:

- Frontend based on Vue 3.
- Backend based on Node.js/Express.
- Redis-backed session mechanisms.
- Containerized deployment with reverse-proxy routing.
- File storage through S3-compatible object storage services.
- Core business data strongly coupled to Airtable.

This architecture enabled fast delivery in earlier stages of product evolution, but it created a concentration of dependencies in external data services and manually maintained infrastructure conventions.

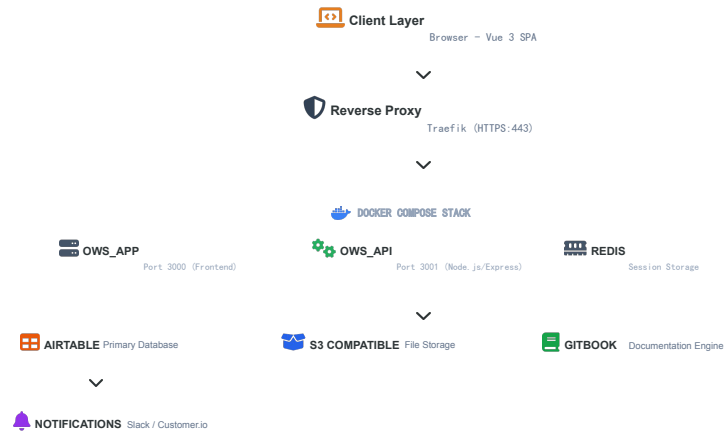


Figure 3.1. Oversight Cloud baseline architecture before the migration, as documented in the pre-migration technical material.

3.2 Airtable as Primary Data Storage

Before the migration to PostgreSQL, Oversight Cloud relied on Airtable as its primary data storage system. Airtable acted not only as a database, but also as an operational platform supporting multiple workflows inside the application domain.

It stored core entities such as users, organizations, sites, simulations, role assignments, and permissions. In addition, Airtable was used for operational forms and for triggering integrations with external services, including Slack notifications.

Unlike a traditional relational DBMS, Airtable organizes data through linked records and lookup fields. This model offered flexibility in the early product phase and enabled rapid prototyping, especially because non-technical stakeholders could inspect and manipulate data through a graphical interface.

3.3 Limitations of Airtable

As the platform evolved and the amount of managed data increased, several limitations became evident.

3.3.1 Data Modeling and Consistency

Relationships between entities (for example organizations, sites, and simulations) were represented through linked records and lookup fields, rather than enforced through explicit foreign-key constraints. As a consequence, consistency guarantees depended heavily on application logic.

During access and synchronization flows, lookup fields could expose heterogeneous representations of the same logical information. In some contexts values appeared as scalars, while in others they appeared as arrays. This increased backend complexity and made query and mapping logic harder to maintain.

3.3.2 Limited Relational Capabilities

Airtable does not provide full relational capabilities comparable to SQL databases. Advanced joins, complex filtering patterns, indexing strategies, and transactional control are more limited than in PostgreSQL-like systems.

3.3.3 Operational Inefficiency

Several workflows depended on manual interactions through Airtable forms. In particular, user creation and onboarding required internal manual intervention, which slowed operations and increased the probability of human error.

3.3.4 Scalability, Coupling, and Cost

Over time, Airtable became both application datastore and operational control surface. This created tight coupling between business logic and Airtable-specific concepts, increasing technical debt and reducing architectural evolvability.

The economic impact was also significant: recurring operational costs reached approximately EUR 1000 per month for database usage alone.

3.4 Motivation for Migration to PostgreSQL

The migration to PostgreSQL was initiated to address these constraints and provide a more robust foundation for the future evolution of Oversight Cloud.

PostgreSQL introduced key advantages:

- explicit relational modeling through primary and foreign keys;
- stronger guarantees for data consistency and integrity;
- improved querying capabilities and indexing mechanisms;
- reduced dependence on Airtable-specific structures;
- lower operational cost exposure;
- improved scalability and maintainability.

The migration also aligned with the broader cloudification strategy and AWS replatforming effort aimed at long-term reliability and maintainability.

Therefore, the objective was not only to replace one storage technology with another, but to progressively transition from a semi-structured operational backend to a maintainable and scalable relational architecture.

Chapter 4

Migration Methodology and Decision Framework

This chapter formalizes the migration methodology used in this thesis. The emphasis is on how decisions were made, not only on which technologies were selected.

4.1 Migration Goals and Success Criteria

The migration goals were defined before implementation to avoid solution bias:

- Preserve operational continuity during transition.
- Reduce architectural coupling to legacy data and identity assumptions.
- Improve deployment reproducibility and environment governance.
- Provide explicit technical and economic justification for major choices.

From these goals, the following success criteria were derived:

- ability to run and validate critical flows during data-layer transition;
- traceable infrastructure definitions across environments;
- measurable reduction of dependency concentration on Airtable;
- migration process auditable through tests, scripts, and CI evidence.

4.2 Candidate Solutions Considered

The decision process considered multiple options for each migration dimension.

4.2.1 Infrastructure and Runtime Options

- Keep existing deployment style with incremental hardening.
- Move to managed cloud services with IaC governance.
- Adopt a full orchestration-first redesign from the beginning.

4.2.2 Alternative Cloud Platforms Considered

During the migration design phase, multiple cloud platform alternatives were considered before selecting the target environment. The comparison focused on scalability, availability of managed services, IaC support, ecosystem maturity, operational complexity, and integration with the existing technical baseline.

Table 4.1. Comparative assessment of cloud provider alternatives for Out-sight Cloud migration

Criterion	AWS	Azure	GCP	On-prem
Managed services fit	High	Medium	Medium	Low
IaC and automation	High	Medium	Medium	Low
Operational complexity	Low to medium	Medium	Medium	High
Ecosystem and maturity	Very high	High	High	Variable
Migration continuity with current baseline	High	Medium-low	Medium-low	Low

Microsoft Azure

Microsoft Azure offers a broad enterprise ecosystem with managed services for compute, storage, databases, networking, and DevOps workflows. Functionally, it provides equivalents for many services used in this project, including:

- Azure Virtual Machines (similar role to EC2),
- Azure Kubernetes Service (AKS),
- Azure SQL Database,
- Azure Blob Storage,
- Azure DevOps.

Azure is particularly strong in Microsoft-centric enterprise contexts. However, for Oversight Cloud it introduced additional migration overhead because the existing deployment strategy and operational knowledge were already aligned with AWS-oriented patterns.

Google Cloud Platform (GCP)

Google Cloud Platform was also considered, especially due to its strengths in containerized workloads and Kubernetes-native operations. Relevant services included:

- Google Kubernetes Engine (GKE),
- Cloud SQL,
- Compute Engine,
- Cloud Storage.

GCP presents strong capabilities in analytics, machine learning, and managed Kubernetes. Nevertheless, AWS was preferred for this migration due to broader service coverage in the specific infrastructure management areas required by the project and stronger alignment with the existing cloudification path.

On-prem Infrastructure

Another option was to continue with self-managed virtual-machine infrastructure (or on-premises style operations). While this approach offers full control over resources and configurations, it significantly increases operational responsibilities:

- infrastructure maintenance,

- scalability management,
- backup and disaster recovery operations,
- monitoring and incident handling,
- high-availability design,
- security hardening and lifecycle management.

This option conflicted with the migration objective of reducing operational overhead through managed cloud services.

4.2.3 Data-Layer Migration Options

- Immediate cutover from Airtable to a relational database.
- Long-term coexistence with no formal migration endpoint.
- Incremental coexistence with progressive parity checks and controlled switchover.

4.2.4 Identity Evolution Options

- Preserve legacy authentication contracts and postpone IAM modernization.
- Introduce standards-based IAM with progressive integration in services.
- Full immediate rewrite of authentication and authorization boundaries.

4.3 Technical and Economic Evaluation Criteria

Each candidate solution was evaluated against a shared set of criteria.

4.3.1 Technical Criteria

- Backward compatibility with existing service behavior.
- Security posture and readiness for standards-based IAM.
- Deployment reproducibility across environments.

- Testability of migration steps and rollback readiness.
- Observability and troubleshooting support during transition.

4.4 Selected Strategy and Rationale

The selected strategy was an incremental migration centered on managed cloud services and reproducible infrastructure definitions:

- AWS-managed building blocks for runtime, routing, storage, and observability.
- Terraform as the infrastructure control plane.
- Progressive data migration path from Airtable to PostgreSQL.
- Validation-oriented coexistence between legacy and target data paths.

AWS was selected because it provided the most complete fit for the project constraints and goals:

- mature IaC integration through Terraform-based workflows,
- managed services required by the target architecture (including ECS, RDS, and S3),
- strong scalability and availability characteristics,
- robust DevOps integration for automated provisioning and updates,
- strong support for cloud-native architecture evolution,
- continuity with the broader company cloudification strategy,
- alignment with the company’s commercial and strategic technology choices.

This strategy was selected because it balances risk, implementation effort, and long-term maintainability. It avoids disruptive full replacement while still creating a clear target-state trajectory.

4.5 Risk Management and Coexistence Plan

Risk management was embedded in the migration design:

- Keep migration steps independently verifiable.
- Separate development and production environment evolution.
- Use dual-backend test paths to detect behavioral divergence early.
- Define data validation artifacts to support switchover decisions.

The coexistence phase is treated as a controlled engineering stage, not as an informal transition period.

Chapter 5

Target Architecture and Solution Design (To-Be)

This chapter presents the target architecture resulting from the decision framework in Chapter 4. The architecture is described as a migration destination designed for incremental adoption.

5.1 Initial Architecture of Outsight Cloud

Before the migration to AWS, Outsight Cloud relied on a containerized architecture deployed through Docker Compose and integrated with multiple external services. This was the initial technical baseline of the platform.

The initial architecture was organized into four layers:

- client layer;
- reverse proxy layer;
- application layer;
- external services layer.

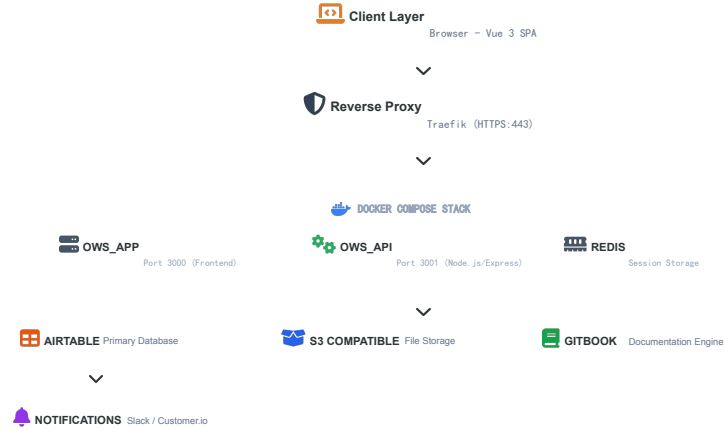


Figure 5.1. Initial architecture of Oversight Cloud before AWS migration.

In this setup, users accessed a Vue.js Single Page Application (SPA) through a web browser. Requests were routed by Traefik and forwarded to application services running in Docker containers. The backend integrated external systems including Airtable, S3-compatible storage, GitBook, Redis, and notification services.

5.1.1 Client Layer

Browser – Vue.js SPA

The frontend entry point was a browser-based SPA implemented in Vue.js. Its responsibilities included:

- rendering the user interface;
- managing user interactions;
- invoking backend APIs;
- displaying simulations, organizations, sites, and documentation.

5.1.2 Reverse Proxy Layer

Traefik Reverse Proxy

Traefik handled incoming HTTPS traffic and routed requests to internal services. Main responsibilities were HTTPS termination, request routing, container discovery, and traffic forwarding. The application was exposed externally through port 443, while internal service communication relied on Docker networking.

5.1.3 Application Layer

The application logic was deployed through a Docker Compose stack with three primary components.

Frontend Container

OWS_APP hosted the Vue frontend and exposed it on port 3000, delivering the web interface and client-side behavior.

Backend Container

OWS_API implemented backend logic with Node.js/Express and exposed APIs on port 3001. It orchestrated users, organizations, sites, simulations, file access, notifications, and external integrations.

Redis

Redis was primarily used as session storage and temporary state support, reducing repeated access to external systems and improving responsiveness.

5.1.4 External Services Layer

Airtable

Airtable was the primary datastore for users, organizations, sites, simulations, permissions, and roles. It also supported operational workflows and integrations.

S3-Compatible Storage

Object storage relied on an S3-compatible service for simulation assets, uploaded files, and project resources, but this storage was not fully integrated into a native AWS architecture.

GitBook Documentation Engine

GitBook was used as an external documentation platform for product documentation, deployment guides, technical manuals, and support material.

Notification Services

Operational notifications depended on external integrations such as Slack and Customer.io for account and workflow events.

5.1.5 Architectural Limitations

Although the initial architecture enabled rapid early development, several limitations emerged:

- strong dependency on Airtable as primary backend;
- limited scalability and weak cloud-native orchestration;
- high coupling with external services;
- manual operational workflows;
- increasing operational complexity over time.

These limitations motivated the migration path toward the AWS architecture presented in the next sections.

5.2 Architecture Overview

The migration of Oversight Cloud to AWS required the definition of a cloud-native architecture able to improve scalability, maintainability, and service integration while preserving the original application workflow.

The selected architecture follows a layered model with four levels:

- client and delivery layer;
- load balancing and routing layer;
- application layer;
- managed services layer.

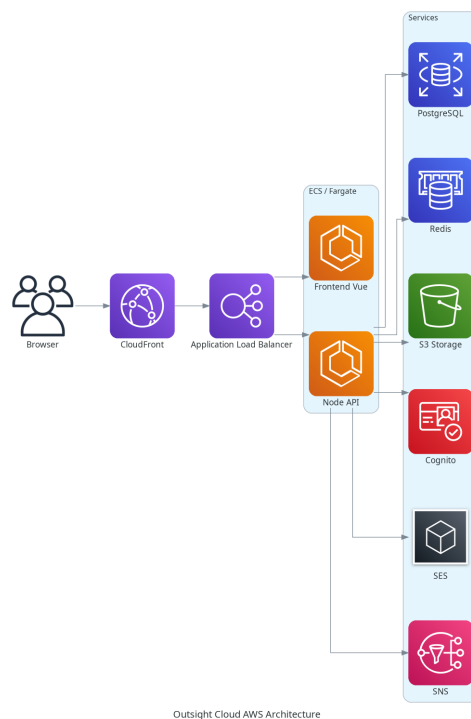


Figure 5.2. Final AWS architecture adopted for Outsight Cloud.

Figure 5.2 shows the left-to-right request flow: users interact through the browser, traffic is delivered through CloudFront and routed by an Application Load Balancer, frontend and backend services run on ECS Fargate, and application logic integrates with managed services including PostgreSQL, Redis, Amazon S3, Cognito, SES, and SNS.

5.3 Client and Delivery Layer

5.3.1 Browser

The browser is the user entry point to Outsight Cloud. Through the frontend interface, users perform simulation visualization, organization and site management, document access, and file upload/download operations. All interactions occur over HTTPS.

5.3.2 Amazon CloudFront

Amazon CloudFront is used as the content delivery layer between users and the internal application infrastructure. Its role is to reduce latency, cache static assets, accelerate delivery, improve availability, and reduce backend load. This is particularly relevant for frontend static resources generated by the Vue application.

5.4 Load Balancing Layer

5.4.1 Application Load Balancer

The Application Load Balancer (ALB) is the central routing component. It receives requests forwarded by CloudFront and distributes traffic across services running on ECS Fargate. It provides request routing, health checks, HTTPS termination, and traffic distribution, enabling independent scaling and improved fault tolerance.

5.5 Application Layer

5.5.1 ECS Fargate Runtime

The application layer is deployed on Amazon ECS Fargate with a serverless container execution model. Compared to traditional VM-based deployment, Fargate removes the need to provision and maintain servers manually.

5.5.2 Frontend Service (Vue)

The frontend service is implemented in Vue.js and is responsible for rendering the user interface, handling user interactions, communicating with backend APIs, and managing authentication-related flows. Deploying it as a dedicated container enables independent release and scaling behavior.

5.5.3 Backend Service (Node.js API)

The backend service is implemented in Node.js and exposes the core APIs used by Oversight Cloud. It orchestrates business logic related to organizations, sites, simulations, authentication, file operations, notifications, and integrations with AWS managed services.

5.6 Managed Services Layer

5.6.1 PostgreSQL

PostgreSQL replaced Airtable as primary data storage. The migration objective was to move from a semi-structured operational backend toward a relational model with stronger consistency and integrity guarantees. Core entities such as users, organizations, sites, and simulations are persisted in PostgreSQL.

5.6.2 Redis

Redis is used as in-memory cache and session storage. It reduces repeated accesses to persistent storage and improves response time for frequently accessed information and temporary state management.

5.6.3 Amazon S3

Amazon S3 provides object storage for simulation files, uploaded assets, documentation artifacts, and deployment-related resources. It was selected for scalability, durability, and native integration with the AWS ecosystem.

5.6.4 Amazon Cognito

Amazon Cognito is used for authentication and identity management, including user registration, login workflows, identity verification, and token handling. This reduces custom authentication logic in the application layer.

5.6.5 Amazon SES

Amazon SES is used for outgoing email communication such as invitations, account-related messages, notifications, and operational communications.

5.6.6 Amazon SNS

Amazon SNS is adopted for asynchronous event-driven notifications. It supports decoupled communication patterns for alerts, internal notifications, workflow events, and service integration triggers.

5.7 Architectural Benefits

Compared to the pre-migration system, the selected architecture provides:

- clearer separation between presentation, business logic, and service layers;
- cloud-native deployment model with managed runtime and services;
- independent scaling of frontend and backend components;
- reduced operational burden through managed infrastructure;
- higher maintainability and better integration with IaC and automation workflows.

Overall, the final AWS design aligns with the broader cloudification strategy of Outsight Cloud and provides a robust baseline for future platform evolution.

Chapter 6

Migration Implementation

This chapter describes the implementation foundations that were already established to execute the migration strategy in a controlled and verifiable way.

6.1 Incremental Execution Plan

The migration was implemented incrementally, with explicit separation between foundation work and full functional cutover. The implementation sequence followed four tracks:

- establish cloud infrastructure foundations on AWS;
- formalize reproducible provisioning with Terraform;
- introduce data migration tooling from Airtable to PostgreSQL;
- implement validation pipelines to check behavioral consistency during coexistence.

This sequencing reduced migration risk by enabling technical validation before irreversible system decisions.

6.2 Infrastructure as Code and Environment Separation

6.2.1 Infrastructure as Code (IaC)

Infrastructure as Code (IaC) is a software engineering approach in which infrastructure is provisioned and managed through code instead of manual configuration procedures [2]. The rationale comes from DevOps principles: infrastructure should be treated with the same rigor applied to application code, including versioning, reviewability, traceability, and repeatability.

Under the IaC paradigm, infrastructure definitions are declarative artifacts stored in version control systems. This replaces fragile practices based on manual console operations, ad-hoc scripts, or outdated runbooks. As highlighted by AWS guidance, declarative infrastructure improves consistency and reliability in environment creation and updates [2].

A key IaC characteristic is that engineers define the desired final state, while automation tooling computes and executes the required operations. This model provides:

- **Consistency:** reproducible environments across development, testing, and production.
- **Version control:** auditable change history for infrastructure evolution.
- **Automation:** integration with CI/CD-driven deployment processes.
- **Reproducibility:** deterministic creation and update workflows.
- **Maintainability:** reusable infrastructure definitions that evolve over time.

In the Outsight Cloud migration, IaC was adopted to automate AWS provisioning and reduce manual intervention during environment setup and updates.

6.2.2 Terraform-Driven Provisioning

Terraform is an Infrastructure as Code (IaC) tool developed by HashiCorp that enables provisioning and lifecycle management of infrastructure through declarative

configuration files [3]. Instead of manually creating resources through cloud consoles, infrastructure is defined as code in human-readable files (HCL), which can be versioned, reviewed, reused, and integrated into software delivery workflows.

Terraform interacts with platforms through providers, i.e., plugins that map Terraform resources to external APIs. In this migration, the AWS provider was used to manage cloud resources including load balancing, container runtime services, networking components, DNS/TLS integration, storage services, and observability-related infrastructure.

6.2.3 Terraform Workflow: Write, Plan, Apply

The Terraform workflow follows three phases [3]:

- **Write:** define the target infrastructure state declaratively in configuration files.
- **Plan:** generate an execution plan that compares desired state and current state, showing which resources will be created, updated, or removed.
- **Apply:** execute the planned operations while respecting resource dependencies through an internal dependency graph.

This workflow was especially useful for migration safety, because planned changes could be reviewed before execution and applied incrementally.

6.2.4 State Management and Team Collaboration

Terraform keeps a state representation of real infrastructure (`terraform.tfstate`) to compute incremental changes instead of recreating complete environments [3]. In practice, state management enabled drift detection and made environment updates predictable across migration iterations.

For collaborative operations, shared state and controlled execution are essential. This improves consistency across development stages and reduces configuration drift during parallel workstreams.

6.2.5 Why Terraform in the Outsight Cloud Migration

Terraform was selected because it aligned with the migration objectives:

- **Standardization:** infrastructure definitions became versioned artifacts.
- **Automation:** provisioning and updates were integrated into CI/CD-oriented processes.
- **Consistency:** environments were reproducible across development and production contexts.
- **Scalability:** infrastructure changes could be applied incrementally.
- **Maintainability:** modular and declarative definitions reduced manual intervention.

Within the Outsight Cloud project, this translated into a reproducible AWS infrastructure baseline supporting the broader cloudification strategy and reducing operational risk during migration.

6.2.6 Environment Governance

A dedicated effort was made to separate development and production concerns. This included environment-specific configuration handling and controlled evolution of shared resources to avoid production side effects during migration experimentation.

6.3 Data Migration Tooling and Validation

6.3.1 Relational Foundation

The migration path from Airtable to PostgreSQL was implemented with schema and migration tooling designed to preserve compatibility constraints. The objective was not only data transfer, but controlled evolution toward explicit schema ownership.

6.3.2 Replication and Validation Scripts

Dedicated scripts were prepared to:

- replicate data from Airtable into PostgreSQL;
- validate consistency through machine-readable reports;
- support migration checkpoints before wider rollout.

These artifacts transform data migration from a one-time operation into a repeatable verification workflow.

6.4 CI/Test Strategy for Dual-Backend Consistency

6.4.1 Dual-Backend Test Design

To support coexistence, smoke tests were designed to run against both data backends (Airtable and PostgreSQL) under a common API expectation. This provided an early warning mechanism for behavioral divergence.

6.4.2 Containerized Validation Environment

The test workflow uses containerized dependencies, including a PostgreSQL service and an Airtable-compatible mock component, so that migration checks can be reproduced reliably in CI.

6.4.3 Pipeline Integration

GitLab CI stages were defined to execute unit and smoke checks across backend variants. This allows migration decisions to be supported by continuous evidence instead of manual spot checks.

6.5 Implementation Scope and Current Status

At the current stage, the implemented work establishes the migration backbone:

- infrastructure foundations and environment governance;
- data replication and validation tooling;
- coexistence-oriented CI/test pipeline.

This scope is sufficient to evaluate migration feasibility and design quality, while leaving room for further functional expansion in subsequent project phases.

Chapter 7

Evaluation and Discussion

This chapter evaluates the migration approach with focus on decision quality, implementation readiness, and risk reduction evidence.

7.1 Validation Methodology

Given the current project stage, evaluation is organized around migration readiness rather than final production performance optimization. The methodology combines:

- architectural consistency checks between migration goals and implemented foundations;
- reproducibility checks through Infrastructure as Code and environment separation;
- coexistence checks through dual-backend testing and CI evidence;
- data migration checks through replication and validation scripts.

7.2 Technical Results

7.2.1 Infrastructure and Deployment Readiness

The migration produced a reproducible infrastructure baseline on AWS, managed through Terraform and integrated with routing, certificate management, container runtime, and observability services.

7.2.2 Data Migration Readiness

The project established concrete mechanisms to compare and validate behavior during Airtable-to-PostgreSQL transition. This is a key technical result because it enables incremental migration decisions based on evidence rather than assumptions.

7.2.3 Testing and Verification Readiness

Dual-backend smoke tests and CI integration provide an automated control surface for migration risk. This increases confidence in backward-compatible evolution during coexistence phases.

7.3 Migration KPIs and Economic Impact

The evaluation framework tracks migration progress using two KPI classes.

7.3.1 Technical Migration KPIs

- percentage of critical flows covered by backend-agnostic smoke tests;
- number of migration checkpoints completed with successful validation reports;
- reproducibility of environment provisioning across development stages;
- incidence of regressions detected before deployment.

7.3.2 Economic Migration KPIs

- reduction in dependency concentration on high-recurring-cost services;
- expected decrease in manual operational effort through IaC and pipeline automation;
- migration effort distribution across infrastructure, data, and application layers.

Initial evidence confirms that the selected strategy is compatible with both technical risk control and economic sustainability goals.

7.4 Limitations and Lessons Learned

7.4.1 Limitations

The current evaluation reflects an intermediate migration stage. Long-term operational behavior and full post-migration cost curves require additional observation after broader adoption.

7.4.2 Lessons Learned

Three lessons emerge clearly:

- migration architecture must be paired with explicit validation architecture;
- coexistence is manageable only when test and data checks are planned from the start;
- technical and economic criteria must be evaluated together, not sequentially.

Chapter 8

Conclusions and Future Work

This thesis addressed the migration of a legacy platform toward a cloud-native architecture as a structured engineering decision problem. The work focused on defining and justifying a migration strategy under technical and economic constraints.

8.1 Key Outcomes

The migration approach demonstrates that legacy-to-cloud-native transition can preserve continuity when it is treated as an incremental engineering process with explicit validation checkpoints.

The decision framework confirms the need to evaluate technical and economic criteria together. Security, compatibility, and operability decisions are sustainable only when recurring cost and operational effort are considered from the beginning.

The implementation evidence shows that coexistence can be managed through dual-backend testing, CI-enforced checks, and data validation artifacts. These elements make migration progress measurable and auditable.

Finally, the produced artifacts (decision rationale, Terraform definitions, migration scripts, validation workflows) improve reusability of the approach beyond a single project context.

8.2 Future Improvements

Future work can extend this thesis in three directions:

- complete and monitor full production cutover stages with long-term KPIs;
- further evolve identity architecture toward full standards-based integration across all services;
- consolidate post-migration cost and reliability analysis with extended operational data.

Overall, the thesis shows that cloud-native migration is most effective when treated as an explicit design process supported by verifiable checkpoints, rather than as a sequence of isolated implementation tasks.

Bibliography

- [1] Amazon Web Services. «What is cloud migration?» Accessed: 2026-05-17. (2026), [Online]. Available: <https://aws.amazon.com/what-is/cloud-migration/>.
- [2] Amazon Web Services. «Infrastructure as code». Accessed: 2026-05-17. (2026), [Online]. Available: <https://docs.aws.amazon.com/whitepapers/latest/introduction-devops-aws/infrastructure-as-code.html>.
- [3] HashiCorp. «What is terraform?» Accessed: 2026-05-17. (2026), [Online]. Available: <https://developer.hashicorp.com/terraform/intro>.